Advanced programming in the Linux environment

Book Proposal

by
Marco Bruno

Proposal contents

Description

Modern Linux systems power everything from cloud infrastructure to embedded devices, yet many developers rely on high-level abstractions without understanding how software actually interacts with the operating system.
Advanced Programming in the Linux Environment provides a structured and practical path to mastering system-level programming on Linux. Rather than functioning as a traditional reference manual, the book transforms low-level system interfaces into a coherent learning journey, guiding readers from fundamental concepts to advanced topics used in real-world software.

The book is built around a simple principle: understanding comes from observing how programs behave at runtime.
To support this, it integrates tools such as *strace* and *gdb* throughout, enabling readers to connect source code with system calls, process behavior, and kernel interaction.

Unlike many existing texts, this book emphasizes modern Linux features and practices, including:

- asynchronous I/O with *io_uring*
- advanced interprocess communication mechanisms
- namespaces and system isolation
- process security with *seccomp*
- system-level authentication through PAM
- in-depth analysis of the ELF format and program execution lifecycle

A defining characteristic of the book is its approach to the Linux manual pages. Instead of treating them as passive documentation, the book turns them into active learning tools, providing multiple real, executable examples for each interface and demonstrating how they are used in practice.

Each chapter combines:

- clear explanations of system interfaces
- carefully designed, runnable code examples
- analysis of runtime behavior using debugging and tracing tools
- dedicated “Theory and Insights” sections to reinforce deeper understanding

Particular attention is given to depth and completeness. Rather than presenting system interfaces superficially, the book examines topics in detail, exploring not only how APIs are used, but also their behavior, design rationale, limitations, and interaction with the Linux kernel and runtime environment. The goal is to provide readers with a comprehensive understanding that extends beyond isolated examples or introductory usage patterns.

The result is a resource that bridges the gap between documentation and practical expertise, helping readers move from writing code to understanding how software operates within the Linux system.

Target

Primary audience:

- Intermediate C programmers who want to develop a solid understanding of Linux system programming
- Software developers transitioning from application-level programming to system-level development

Secondary audience:

- Backend and systems engineers working on performance-critical or low-level components
- Advanced students in computer science or software engineering

Prerequisites:

- Solid knowledge of C programming
- Basic familiarity with Linux command-line usage

Key Features

- **Modern focus**
  Covers recent Linux features such as *io_uring*, namespaces, and advanced kernel interfaces often missing from traditional books.
- **From documentation to mastery**
  Transforms Linux manual pages into structured, example-driven learning rather than passive reference material.
- **Runtime-oriented learning**
  Strong emphasis on observing program behavior using tools like *strace* and *gdb*, helping readers build mental models of system execution.
- **Complete system-level perspective**
  Covers the full lifecycle of a program: loading, execution, interaction with the kernel, and termination.
- **Balanced theory and practice**
  Each chapter includes a “Theory and Insights” section to deepen understanding beyond code examples.
- **Real-world applicability**
  Focus on practical patterns and techniques used in actual system and backend development.

Why this book

This book addresses a gap between traditional system programming references and the needs of modern developers.
Classic texts in this area provide strong theoretical foundations but often:

- lack coverage of newer Linux features
- are structured primarily as reference manuals
- do not emphasize runtime analysis and debugging workflows

At the same time, many online resources are fragmented, incomplete, or overly simplified.
Advanced Programming in the Linux environment positions itself as:

- a structured learning path rather than a reference
- a modern complement to classic texts
- a bridge between documentation and real-world usage

It is particularly suited for developers who want not only to use system interfaces, but to understand how software behaves within the Linux environment.

The need for system-level understanding is increasing due to:

- widespread use of Linux in cloud and containerized environments
- growing importance of performance, scalability, and security
- renewed interest in low-level programming and systems design

At the same time, developers often rely on abstractions that hide system behavior, creating a gap between writing code and understanding its execution.
This book addresses that gap by providing a modern, practical, and deeply technical guide to Linux system programming.

About the authors

Marco Bruno

Marco Bruno is a software developer and educator with over twenty years of experience in information technology and professional training. He began his career as a system administrator, working extensively with UNIX and Linux systems, where he developed strong expertise in system-level operations.

Over time, he specialized in C programming on Linux, focusing on system programming and low-level software development. He has taught these subjects through private courses and technical training programs, combining practical experience with a clear and methodical teaching approach. He has also gained significant international working experience, collaborating with companies and professionals across different countries and technical environments. This exposure has contributed to a broad and adaptable perspective on software development practices and system architectures.

Throughout his career, Mr. Bruno has developed software for a wide range of companies, working across multiple domains, including system software, user applications, network programming, and web development. His experience spans several widely used programming languages, including C, Python, and Java.
He combines extensive industry experience with a strong ability to both develop complex software systems and effectively teach their underlying principles.

Table of contents

Chapter 0 – Tools

Chapter I – File I/O

Chapter II – Files and directories

Chapter III – Environment and system databases

Chapter IV – Memory

Chapter V – Processes

Chapter VI – Interprocess Communication and Terminals

Annotated table of contents

Chapter 0 — Tools

This introductory chapter presents the essential tools commonly used for software development in a Linux environment, along with an overview of the standards that define software behavior and portability. It is intended to provide readers with the necessary background for the topics covered in the rest of the book; those already familiar with these concepts may choose to skip it.

The chapter covers the compilation process and the use of key development tools such as *gcc*, *make*, and *autoconf*, as well as debugging with *gdb*. It also introduces the source code and how it can be obtained and organized in a Linux system. In addition, it outlines the most widely used software standards at a high level and includes a simple introduction to the GNU General Public License (GPL) and its role in software development and distribution.

Chapter 1 — File I/O

This chapter introduces file input/output operations, starting from fundamental system calls such as *open*, *read*, *write*, and *close*, and progressing to more advanced mechanisms. Topics include file truncation and appending, file descriptor duplication, I/O multiplexing, buffering strategies, asynchronous I/O (both POSIX and Linux-specific), file locking, and interprocess communication through pipes and FIFOs.

Chapter 2 — Files and Directories

This chapter examines the APIs used to interact with the filesystem. It covers file metadata (*stat*), links, permissions, and inode structures, as well as operations on directories such as creation, deletion, and renaming. Additional topics include device files, Linux capabilities, filesystem organization, quotas, and resource limits.

Chapter 3 — Environment and system Databases

This chapter explores the process environment and its interaction with the operating system. It includes functions for handling time and date, environment variables, and localization (locale). It also provides a detailed overview of system databases, such as user and group information, along with login records and system identification (*uname*). The chapter concludes with a dedicated section on Pluggable Authentication Modules (PAM).

Chapter 4 — Memory

This chapter provides an in-depth analysis of memory management in Linux. It covers dynamic memory allocation, memory mapping, memory protection, and locking mechanisms. It also examines dynamically loaded objects and presents a detailed study of the ELF (Executable and Linkable Format), including how programs are loaded into memory and how processes start and terminate.

Chapter 5 — Processes

This chapter focuses on process management and multithreading. It covers process creation and control through *fork*, *clone*, *execve*, and *wait*, as well as security features such as *seccomp*. Additional topics include process groups, sessions, terminal control, signal handling, timers, thread programming, and daemon processes, all supported by practical examples.

Chapter 6 — Interprocess Communication and Terminals

This chapter covers the main IPC mechanisms available in Linux, including POSIX message queues, semaphores, shared memory, and sockets. It also explores UNIX domain sockets in detail, along with terminal and pseudoterminal interfaces, providing numerous examples to illustrate their use in real-world scenarios.

Sample Chapters

Chapter I

File I/O

I.1 Reading files

In Linux, almost everything is represented as a file: the keyboard, hard disks, terminals, printers, and even communication devices are exposed through the filesystem interface. Regular files are also represented in the same way. Files can be opened to perform read and/or write operations, and each file has associated attributes such as size, access permissions, ownership, and type.
Like most Unix-like operating systems, Linux defines three standard streams as the primary input/output interfaces for programs: *standard input*, *standard output*, and *standard error*. In Linux, these streams are represented as file descriptors and are identified by both a name and a numeric value.

`stdin`	`0`	Standard input
`stdout`	`1`	Standard output
`stderr`	`2`	Standard error

Tab1. The main I/O devices

When we execute a command from the shell, its output is normally displayed on the *standard output*, while error messages are sent to the *standard error* stream. *Standard input* represents the source from which data is read, typically the keyboard. *Standard output* may correspond to the terminal, a printer, or a file used to store the results of program execution. Likewise, *standard input* and *standard error* can also be redirected to files or other devices instead of the console.

I.1.1 Opening files: `open` and `close`

The `open`[1] function opens the file specified by its first argument according to the mode indicated by the second argument, and returns a *file descriptor*. A file descriptor is a positive integer used by the operating system to identify an open file within a program. Every file opened by a process in the system is associated with a file descriptor.

```
#include <fcntl.h>

int open(const char *pathname, int flags);
```

1 See the `open(2)` manual page

```
                                        Returns: the new file descriptor, -1 on error
```

The `openfile.c` program provides an example of how this works.

```
1 #include <stdio.h>
2 #include <fcntl.h>

3 int main (int argc, char *argv[])
4 {
5     int fd;
6     int fd2;
7     int fd3;

8     fd = open("file.txt", O_RDONLY);
9     fd2 = open("file2.txt", O_WRONLY);
10    fd3 = open("file3.txt", O_RDWR);
11    printf("fd = %d", fd);
12    printf("\nfd2 = %d", fd2);
13    printf("\nfd3 = %d", fd3);
14    return 0;
15 }
```

openfile.c Opens three files in different modes

Line 1. Includes the header file for standard I/O functions
Line 2. Includes the header file required for the `open` function
Lines 5–7. Declare three file descriptors
Line 8. Opens the file in read-only mode
Line 9. Opens the file in write-only mode
Line 10. Opens the file in read/write mode
Lines 11–13. Print the values of the file descriptors

We create two files and then run the program.

```
[linux@io] touch file.txt file2.txt
[linux@io] ./openfile
fd = 3
fd2 = 4
fd3 = -1
```

The value of the third descriptor is `-1` because the file we attempted to open does not exist. The first descriptor has the value `3` because the descriptors `0`, `1`, and `2` are already reserved for `stdin`, `stdout`, and `stderr`, respectively. For each additional file opened, the operating system typically assigns the next available file descriptor. Therefore, if `file3.txt` existed and were opened successfully, its descriptor would likely be `5`, and so on.

The values for the `flags` argument are listed in the following table.

`O_RDONLY`	Read only
`O_WRONLY`	Write only
`O_RDWR`	Read/Write
`O_APPEND`	Append

Tab 2. Opening modes (`open`)

The `close`[2] function closes the file associated with the file descriptor passed as an argument.

```
#include <unistd.h>

int close(int fd);

                                        Returns: 0 on success, -1 on error
```

The `closefile.c` program shows an example.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>

4 int main (int argc, char *argv[])
5 {
6     int fd;
7     open("file.txt", O_RDONLY);
8     close(fd);
9     return 0;
10 }
```

closefile.c Open and close a file

Line 3. Includes the header file required for the `close` function
Line 6. Declare a descriptor
Line 7. Opens the file in read-only mode
Line 8. Closes the file associated with the `fd` descriptor

We execute the program

```
[linux@io] ./closefile
[linux@io]
```

2 See the `close(2)` manual page

The program opens and then closes the file. `close` can return one of the following errors[3]

EBADF	The descriptor is not a valid descriptor
EINTR	The call was interrupted by a signal
EIO	I/O Error
ENOSPC	Not enough space on the disk
EDQUOT	The user quote on fs is finished

Tab 3. `close` errors

I.1.2 Reading from file: `read`

The `read`[4] function reads a count of characters from the file descriptor `fd` and stores them in `buf`.

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);

                                Returns: the number of bytes read, -1 on error
```

When `read` is invoked, it returns the number of bytes actually read. In some cases, this value may be smaller than the number of bytes requested, for example, when the *end of file* is reached before the requested amount has been read, when reading from a terminal device, a network device, or a pipe, or when the operation is interrupted by a signal.
After a successful call, the file offset is automatically increased by the number of bytes read.

The `readfromfile.c` program reads from a file.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>

4 #define COUNT 128
5 int main (int argc, char *argv[])
6 {
7     int fd;
8     int n;
9     char buffer[COUNT];

10    fd = open("./file.txt", O_RDONLY);
11    if (fd < 0) {
12          printf("Open error - exit..");
```

3 For a complete list see the `close(2)` manual page
4 See the `read(2)` manual page

```
13          return 1;
14      }

15      n = read(fd, buffer, COUNT);
16      printf("Bytes read: %d", n);
17      printf("\nContent: %s", buffer);
18      close(fd);
19      return 0;
20  }
```

---

readfromfile.c Reads the content of the file

Line 4. Constant defining the number of bytes to read from the file
Line 7. File descriptor
Line 8. Integer variable that stores the number of bytes actually read
Line 9. Array of 128 characters used as a buffer to store the data read from the file
Line 10. Opens the file
Line 11. Checks whether `open` returned `-1` (indicating failure to open the file)
Lines 12–13. Prints an error message and exits the program
Line 15. Attempts to read 128 bytes from the file and store them in the buffer
Line 16. Prints the number of bytes read
Line 17. Prints the contents of the buffer
Line 18. Closes the file

We write some data into the `file.txt` file and then run the program.

```
[linux@io] echo Linux > file.txt
[linux@io] ./readfromfile
Readed bytes: 6
Content: Linux
```

The buffer size is 128 bytes, but the file contains only 6 bytes. The `read` function reads data from the file up to the specified buffer size (128 bytes). This means it will return up to 128 bytes if available; however, since the file is smaller, only 6 bytes are actually read, and no additional data is returned beyond the end of the file.

I.1.3 Writing to file: `write`

The `write`[5] function is used to write data to a file descriptor. It writes up to `count` bytes from the buffer `buf` to the file descriptor `fd` and returns the number of bytes actually written.

```
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);

                                Returns: the number of written bytes, -1 on error
```

For the *standard input*, *standard output* and *standard error*, the corresponding constants are defined in the `unistd.h` header file. These constants refer to their respective file descriptors.

STDIN_FILENO	0
STDOUT_FILENO	1
STDERR_FILENO	2

Tab 4. Standard descriptors

The `readfile.c` program opens a file, repeatedly reads from it until at least one byte is obtained, prints the number of bytes read along with the data, and then closes the file.

```
1  #include <stdio.h>
2  #include <fcntl.h>
3  #include <unistd.h>

4  #define COUNT 128

5  int main()
6  {
7      int fd;
8      char buffer[COUNT];

9      fd = open("./file.txt", O_RDONLY);
10     if (fd < 0) {
11         printf("Open error - exit..");
12         return 1;
13     }

14     while(read(fd, buffer, COUNT) > 0)
15         write (STDOUT_FILENO, buffer, COUNT);
16     close(fd);
```

5 See the `write(2)` manual page

```
17    return 0;
18 }
```

---

readfile.c Reads a file a certain number of bytes at a time

Line 3. Includes the header file for the `read` function
Line 4. Defines the constant `COUNT` with value 128
Line 7. File descriptor
Line 8. Character array of 128 bytes used as a buffer
Line 9. Opens the file
Line 10. Checks if `open` returns `-1`
Lines 11–12. Prints an error message and exits the program
Line 14. Loop that reads 128 bytes at a time until at least one byte is read
Line 15. Prints the number of bytes read
Line 16. Closes the file

We execute the program

```
[linux@io] ./readfile
Linux
```

Unlike the `readfromfile.c` program, the `readfile.c` program reads from the file until data is available. Each call to `read` retrieves up to 128 bytes from the file. Let's run both programs using a file larger than 128 bytes.

```
[linux@io] cp /etc/passwd file.txt
[linux@io] ./readfromfile
Bytes read: 128
Content: io:1000:1000::/home/io:/usr/bin/zsh
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

[linux@io] ./readfile
io:1000:1000::/home/io:/usr/bin/zsh
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
...output...
```

The `readfromfile.c` program reads 128 bytes from the file, while the `readfile.c` program continues reading the entire file in chunks of 128 bytes (the size of the buffer). The size of `buf` can be greater than `count`, but `write` will still write only `count` bytes. The data type `ssize_t`[6] is used for counting bytes and is defined in the `sys/types.h` and `unistd.h` header files.

6 See the `system_data_types(7)` manual page

I.1.4 The standard library functions: `fopen`, `fclose` and `fcloseall`

The `open`, `read`, `close`, and `write` functions are *system calls*. The *standard library* provides a set of file I/O functions that allow files to be handled as *streams*. These functions are not Linux system calls; they belong to the *C standard library* and are defined in the `stdio.h` header file.

The `fopen`[7] function is used to open a file and associate it with a *stream*. It takes as arguments the file name and the opening mode, and returns a pointer to the corresponding *file stream*. The `fclose`[8] function closes the file associated with the stream passed as its argument.

```
#include <stdio.h>

FILE *fopen(const char *pathname, const char *mode);

                                    Return: a pointer to the file, NULL on error

int fclose(FILE *stream);

                                                Returns: 0 on success, EOF on error
```

When working with multiple open files, it may be useful to close all streams at once. The `fcloseall`[9] function closes all streams opened by the process.

```
#include <stdio.h>

int fcloseall(void);

                                                Returns: 0 on success, EOF on error
```

7 See the `fopen(3)` manual page
8 See the `fclose(3)` manual page
9 See the `fcloseall(3)` manual page

I.1.5 Reading one character at a time: `fgetc` and `fputc`

The `fgetc`[10] and `fputc`[11] functions are used to read and write a single character, respectively. The first reads a character from the specified input stream and returns it. The second writes a character to the specified output stream and returns the written character.

```
#include <stdio.h>

int fgetc(FILE *stream);
                                        Returns: the read character, EOF on error

int fputc(int c, FILE *stream);
                                      Return: the written character, EOF on error
```

The `readfilef.c` program reads the contents of the file and prints them to the screen.

```
1 #include <stdio.h>

2 int main()
3 {
4     FILE *file;
5     char c;

6     file = fopen("./file.txt", "r");
7     while (c!=EOF) {
8           c=fgetc(file);
9           if(c!=EOF)
10                fputc(c, stdout);
11    }
12    fclose(file);
13    return 0;
14 }
```

readfilef.c Read one character at a time

Line 4. Declaration of a `FILE` pointer
Line 5. Declaration of a `char` variable
Line 6. Opens the file
Line 7. Loop that continues until the character read from the file is equal to `EOF` (End of File)
Line 8. Reads one character from the file
Line 9. Checks whether the character is not `EOF`
Line 10. Prints the character
Line 10. Closes the file

10 See the `fgetc(3)` manual page
11 See the `puts(3)` manual page

The program opens the file, loops through its contents reading one character at a time, prints each character to the screen, and continues until it reaches the end of the file, after which it closes the file.

file.txt

```
Linux Programming
1234567890
```

The program is executed

```
[linux@io] ./readfilef
Linux Programming
1234567890
```

The `FILE` data type is an object used to represent streams, and it is defined in the `stdio.h` header file.
The standard library also defines constants in `stdio.h` for standard input, standard output, standard error, and end-of-file handling.

`stdin`	`0`
`stdout`	`1`
`stderr`	`2`
`EOF`	`-1`

Tab 5. The standard streams

We present the same program using a more concise syntax.

```
1 #include <stdio.h>

2 int main()
3 {
4     char c;
5     FILE *fp;
6     fp = fopen("./file.txt", "r");
7     while ((c=fgetc(fp)) != EOF)
8           fputc(c, stdout);

9     fclose(fp);
10    return 0;
11 }
```

readfilefs.c

The program is executed

```
[linux@io] ./readfilefs
Linux Programming
1234567890
```

The `fopen` function accepts the following flags

`r`	Read
`r+`	Read and Write
`w`	Write. If the file exists its contents are truncated to zero
`w+`	Read and Write. If the file does not exist it is created
`a`	Writes at the end of the file (appends to existing content)
`a+`	Reading and queuing. If the file does not exist it is created

Tab 6. File opening modes of `fopen`

In addition to the standard flags `fopen` also accepts the following

`c`	The file opens with deletion disabled
`e`	The descriptor will be closed if one of the `exec`[12] functions is used (corresponds to the `FD_CLOEXEC`)
`m`	The file is opened and accessed via `mmap`[13]
`x`	Open the file exclusively

Tab 7. `fopen` flags

The correspondence between the `fopen` and `open` flags is shown in the following table.

fopen	**open**
`r`	`O_RDONLY`
`w`	`O_WRONLY | O_CREAT | O_TRUNC`
`a`	`O_WRONLY | O_CREAT | O_APPEND`
`r+`	`O_RDWR`
`w+`	`O_RDWR | O_CREAT | O_TRUNC`
`a+`	`O_RDWR | O_CREAT | O_APPEND`

Tab 8. File opening methods (`open` and `fopen`)

12 See the `exec(3)` manual page

13 See the `mmap(2)` manual page

The `stdin2stdout.c` program copies data from standard input to the standard output, one character at a time.

```
1 #include <stdio.h>

2 int main()
3 {
4     char c;

5     while(c=fgetc(stdin))
6           fputc(c, stdout);
7 }
```

stdin2stdout.c Copy `stdin` to `stdout`

The program reads one character at a time from standard input (`stdin`) and prints it to standard output (`stdout`), effectively echoing the characters typed by the user onto the screen.

```
[linux@io] ./Stdin2Stdout
hello
hello
Linux
Linux
```

I.1.6 Reading one line at a time: `fgets` and `fputs`

The `fgets`[14] function reads a line from the file passed as an argument and stores it in a buffer. It takes as parameters the buffer where the line will be stored, the maximum number of bytes to read, and the file stream from which to read, and it returns the resulting string.
The `fputs`[15] function writes a line to the file passed as an argument. It takes as parameters the buffer containing the data to be written and the output file stream.

```
#include <stdio.h>

char *fgets(char *s, int size, FILE *stream);

                                        Return: the buffer read, NULL on error

int fputs(const char *s, FILE *stream);

                                    Return: a non negative integer, EOF on error
```

The `readfileline.c` program uses `fgets` to read the file one line at a time.

```
1 #include <stdio.h>

2 int main()
3 {
4     FILE *file;
5     char buffer[1024];
6     int count = 1024;

7     if ((file = fopen("./file.txt", "r")) == NULL) {
8           printf ("Error opening file");
9           return 1;
10    }

11    while (fgets (buffer, count, file))
12          fputs (buffer, stdout);

13    fclose (file);
14    return 0;
15 }
```

readfileline.c Read the file one line at a time

Line 4. Declares a file pointer
Line 5. Declares a 1024-byte character array

14 See the `fgetc(3)` manual page
15 See the `puts(3)` manual page

Line 7. Opens the file in read mode
Lines 8–9. If the file cannot be opened (`fopen` returns `NULL`), it prints an error message and exits the program
Line 11. Loops through the file; `fgets` reads up to the specified number of bytes and stores them in the buffer
Line 12. Prints the buffer to standard output
Line 13. Closes the file

This version of the program uses `fopen` to open the file, `fgets` to read from the stream, and `fputs` to output the read data to the screen.

```
[linux@io] ./readfileline
Hello world

1234567890
```

The program reads the file one line at a time and prints it to the screen. The `fgets` function attempts to read up to `count - 1` bytes from the stream; if it encounters an end-of-file (`EOF`) or a newline character, it stops reading.

The `stdin2stdoutline.c` program copies standard input to standard output, line by line.

```
1 #include <stdio.h>

2 int main()
3 {
4     char buf[256];
5     while(fgets(buf, 256, stdin) != NULL)
6           fputs(buf, stdout);
7 }
```

stdin2stdoutline.c Copy `stdin` to `stdout` one line at a time

Line 5. `fgets` attempts to read up to 256 bytes from standard input until it encounters an `EOF` or a newline character, and stores the data in the buffer `buf`.
Line 6. `fputs` writes the contents of `buf` to standard output.

The program is executed

```
[linux@io] ./stdin2stdoutline
hello Linux
hello Linux
```

The program copies each input line to standard output.

Chapter I continues…..

Chapter II

Files and directories

II.1 Files

As mentioned in the previous chapter, on a Linux operating system everything is a file. A file is a structure that contains data and information about that data. In the C Library there are functions that allow you to read the information of the files and others that allow you to modify the properties of the files. In Linux there are different types of files, e.g. to manage a device (terminal, keyboard, screen, etc..) and to represent it in the file system it is used a different type of file than the one used to represent a simple text (or data) file. Each file in a Linux file system belongs to a particulat user and group, has a creation date and a last modification date and has a set of permissions that define who and how can access the file and what operation can be performed on it (read, write, execute). A directory is a point of access in the file system and within it there can be files or directories. However a directory also is a file like everything else in a Linux fs.

II.1.1 Stat of files: `stat`, `fstat` and `lstat`

To retrieve the information about a file the following functions are used.

```
#include <sys/stat.h>

int stat(const char *restrict pathname, struct stat *restrict statbuf);

int fstat(int fd, struct stat *statbuf);

int lstat(const char *restrict pathname, struct stat *restrict
          statbuf);

#include <fcntl.h>
#include <sys/stat.h>

int fstatat(int dirfd, const char *restrict pathname, struct stat
            *restrict statbuf, int flags);

                                    Return: on success 0, on error -1 and errno
```

All this functions return information about a file in a buffer called `statbuf`. To retrieve information about a file the program must have the permission to traverse all the directories of the path where the file reside. Permissions required are those of execution because to traverse a

directory the user or the program need to ‘execute’ that directory.[16] `stat`, `fstatat` and `lstat` need these permissions to reach the file and read the metadata.
`stat` and `fstatat` retrieve the information about the file pointed by `pathname`
`lstat` is identical to stat but `pathname` is a symbolic link so it return information about the link not the file pointed by the link
`fstat` is identical to `stat` except that the file is specified by a file descriptor (`fd`) instead of the pathname
All the functions return the information in a `stat`[17] structure (`statbuf`). Since the goal of theese functions is to retrieve information about a file it is necessary for the structure that contains the information and for the pathname to be accessible only by one pointer, this is the reason why `pathname` and `statbuf` are declared `restrict`[18]. The argument `dirfd` in `fstatat` specifies the way to calculate the path of the file based on the following rules:

a. if the pathname in `pathname` is relative then is interpreted relative to the directory specified in `dirfd` rather than relative to the current work directory
b. if `pathname` is relative and `dirfd` has the special value `AT_FDCWD` then `pathname` is interpreted relative to the current working directory.
c. If the pathname specified in `pathname` is absolute `dirfd` is ignored.

The argument `flags` can `0` or one of the following values:

`AT_EMPTY_PATH`	If `pathname` is an empty string the function will operate on the file referred to by `dirfd`. This last in this case can refer to any type of file. If `dirfd` is `AT_FDCWD` the function operate on the current working directory.
`AT_NO_AUTOMOUNT`	Don’t automount the terminal (“basename”) component of `pathname`
`AT_SYMLINK_NOFOLLOW`	If `pathname` is a symbolic link do not dereference it and return information on the link itself

Tab 1. Values of the `flags` argument of `fstatat`

The possible errors returned in `errno` are the following

`EACCESS`	search permission is denied for one of the directories in the path
`EBADF`	`fd` is not a valid file descriptor
`EBADF (fstatat)`	`pathname` is relative but `dirfd` is not `AT_FDCWD` nor a valid descriptor
`EFAULT`	bad address

16 To execute a directory means literally ‘go inside the directory’, to read a directory means ‘read the content of the directory’
17 See the `stat(3)` manual page
18 About the use of the keyword `restrict` consult the *C/99 Standard*, the link is in the bibliograpy

EINVAL (fstatat)	flags has not a valid value
ELOOP	too many symbolic links encountered while traversing the path
ENAMETOOLONG	pathname is too long
ENOENT	a component of pathname does not exist or is a not valid symbolic link
ENOENT (fstatat)	pathname is an empty string and AT_EMPTY_PATH is not specified
ENOMEM	out of memeory
ENOTDIR	a component of pathname is not a directory
ENOTDIR (fstatat)	pathname is relative and dirfd is a file descriptor referring to a file other than a directtory
EOVERFLOW	pathname or fd refers to a file whose size, inode number, or number of blocks cannot be represented in, respectively, the types off_t, ino_t, blkcnt_t[19]

Tab 2. errno values for the stat functions

The information retrieved from the file are stored in the structure of type stat[20]. This structure represent the file status.

```
#include <sys/stat.h>

struct stat {
   dev_t st_dev;              /* ID of device containing file */
   ino_t st_ino;              /* Inode number */
   mode_t st_mode;            /* File type and mode */
   nlink_t st_nlink;          /* Number of hard links */
   uid_t st_uid;              /* User ID of owner */
   gid_t st_gid;              /* Group ID of owner */
   dev_t st_rdev;             /* Device ID (if special file) */
   off_t st_size;             /* Total size, in bytes */
   blksize_t st_blksize;      /* Block size for filesystem I/O */
   blkcnt_t st_blocks;        /* Number of 512 B blocks allocated */

   /* Since POSIX.1-2008, this structure supports nanosecond
      precision for the following timestamp fields */

   struct timespec st_atim;       /* Time of last access */
   struct timespec st_mtim;       /* Time of last modification */
   struct timespec st_ctim;       /* Time of last status change */
```

19 See the stat(2) manual page for an example of this error and the manual pages: off_t(3type), stat(3type), blkcnt_t(3type) for the types

20 See the stat(3type) manual page

```
   #define st_atime st_atim.tv_sec /* Backward compatibility */
   #define st_mtime st_mtim.tv_sec
   #define st_ctime st_ctim.tv_sec
};
```

The `stat` structure is composed by the following fields:

`st_dev`	device on which the file resides
`st_ino`	the inode[21] number of the file
`st_mode`	type and mode of the file
`st_nlink`	number of hard links to the file
`st_uid`	user ID of owner of the file
`st_gid`	group ID of the owner of the file
`st_rdev`	device represented by the file
`st_size`	size of the file
`st_blksize`	block size
`st_blocks`	number of blocks allocated to the file (1 block = 512 bytes)
`st_atime`	time of last access to the file data
`st_mtime`	time of last modification of file data
`st_ctime`	file's last status change timestamp

Tab 3. Fields of the `stat` structure

The other fields (`st_atim.tv_sec, st_mtim.tv_sec, st_ctim.tv_sec`) are used for backward compatibility. The field `st_ino` represent the inode number of the file. The inode is a number that identifies the file in the file system. Each file or directory in the file system has his inode. On the Linux systems, to see the information about a file is available the program `stat`. When we create a symbolic link to a file we create a pointer to the inode number of that file.

Let's create a file and write some text inside it

```
[linux@fidir] touch file.txt
[linux@fidir] echo 'hello' > file.txt
```

Then create a symbolic link to the file with the command `ln` and the option `-s`

```
[linux@fidir] ln -s file.txt file2
[linux@fidir] ls -l
total 4
```

21 An inode is a number that identifies a file on the file system, each file and directory has his unique inode number

```
lrwxrwxrwx  1 fidir fidir      8 Mar 23 19:58  file2 -> file.txt
-rw-r--r--  1 fidir fidir      6 Mar 23 19:58  file.txt
```

As we can see the link `file2` point to the file `file.txt`, if we execute `cat` on `file2` we see the content of `file.txt`

```
[linux@fidir] cat file2
hello
```

To see the information about the file let's use the `stat` command

```
[linux@fidir] stat file.txt
  File: file.txt
  Size: 6         Blocks: 8         IO Block: 4096   regular file
Device: 259,2 Inode: 23364606    Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   fidir)   Gid: ( 1000/   fidir)
Access: 2024-03-23 20:01:37.452040888 +0700
Modify: 2024-03-23 19:58:18.396035269 +0700
Change: 2024-03-23 19:59:46.536037757 +0700
 Birth: 2024-03-23 19:58:11.664035079 +0700
```

In the firs three lines of the output we can see the size, the number of the blocks, the type of the file, the device, the inode and the number of links pointing to this file

Let's execute the `stat` command on the symbolic link `file2`

```
[linux@fidir] stat file2
  File: file2 -> file.txt
  Size: 8         Blocks: 0         IO Block: 4096   symbolic link
Device: 259,2 Inode: 23364884    Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/   fidir)   Gid: ( 1000/   fidir)
Access: 2024-03-23 19:59:54.840037991 +0700
Modify: 2024-03-23 19:58:38.512035836 +0700
Change: 2024-03-23 19:59:46.536037757 +0700
 Birth: 2024-03-23 19:58:38.512035836 +0700
```

The inode numbers are different because a symbolic link is treated as a separate file in the filesystem, with its own inode. A symbolic link acts as a pointer to another file, but it is still considered an independent filesystem object.
If inode `23364884` is deleted, the symbolic link (`file2`) is removed, while `file.txt` remains unaffected. On the other hand, if inode `23364606` is deleted, `file.txt` is removed, and `file2` becomes a broken symbolic link that points to a non-existent file.

The `Links` field in the output does not refer to symbolic links; it refers to *hard links*. A *hard link* is a reference to a file that, unlike a symbolic link, is not a separate filesystem entity. Multiple hard links can point to the same *inode*, meaning a single inode number may correspond to multiple filenames.

For this reason, a symbolic link has its own inode number, different from the target file, while a hard link shares the same inode number as the original file.
We now create a hard link to the file `file.txt` (command `ln` without options) and execute the `stat` command.

```
[linux@fidir] ln file.txt hfile3
[linux@fidir] stat hfile3
  File: hfile3
  Size: 6         Blocks: 8         IO Block: 4096   regular file
Device: 259,2 Inode: 23364606    Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/   fidir)   Gid: ( 1000/   fidir)
Access: 2024-03-23 20:01:37.452040888 +0700
Modify: 2024-03-23 19:58:18.396035269 +0700
Change: 2024-03-23 20:08:06.860051880 +0700
 Birth: 2024-03-23 19:58:11.664035079 +0700
```

The inode number of `hfile3` is the same as that of `file.txt`, and number of links is 2 because the same inode is associated with two filenames: `file.txt` and `hfile3`. However, if we execute `ls -l`, we can see that `hfile3` appears as a separate file and does not point to `file.txt`.

```
[linux@fidir] ls -l
total 8
lrwxrwxrwx 1 fidir fidir 8 Mar 23 19:58 file2 -> file.txt
-rw-r--r--  2 fidir fidir 6 Mar 23 19:58 file.txt
-rw-r--r--  2 fidir fidir 6 Mar 23 19:58 hfile3
```

In reality, `hfile3` is not a different file, but another reference to the same file. Even though it appears as a separate file in the output of `ls`, it is simply another filename used to access the same underlying data and inode.

We add some text to `file.txt` and look at the content of `hfile3`

```
[linux@fidir] echo 'world' >> file.txt
[linux@fidir] cat hfile3
hello
world
```

When we write in `file.txt` we are effectively telling Linux: 'Write this text into the file identified by inode number 23364606". Since this inode has two references (two filenames that refer to the same data), Linux updates the data associated with that inode, and both references will then access the updated content.
If we execute `cat` on the symbolik link `file2`, it will display the contents of `file.txt`. However, `file2` is not a direct reference to `file.txt` in the same way a hard link is. Instead, `file2` is a separate file whose inode contains a pointer to the path of another file. In other words, `file2` is itself a filesystem object (with its own inode), and that inode stores information that directs the operating system to `file.txt`. Therefore, unlike a hard link, a symbolic link does not share the same inode or data as the target file.

The following picture shows this relationship.

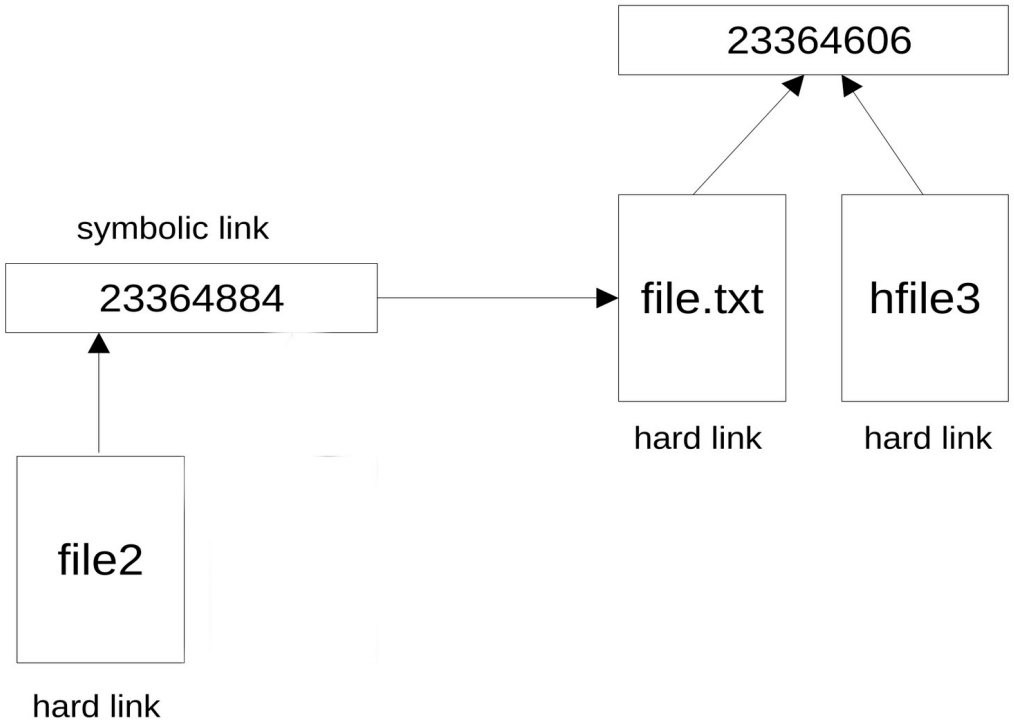

Fig 1. Relationship between hard links and symbolic links

`file2` is an hard link to the inode 23364884 that is a symbolic link to `file.txt` that is an hard link to the inode 23364606. `hfile3` is another hard link to the same inode. We can also create another hard link to the inode 23364884.

```
[linux@fidir] ln file2 sfile4
[linux@fidir] ls -l
total 8
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 file2 -> file.txt
-rw-r--r--      2 fidir fidir 6 Mar 23 20:38 file.txt
-rw-r--r--      2 fidir fidir 6 Mar 23 20:38 hfile3
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 sfile4 -> file.txt
```

`sfile4` is another symbolic link to `file.txt`, this because we create an hard link that is a reference to an inode that points to another file (`file.txt`).
Let's execute `stat` on `sfile4`

```
[linux@fidir] stat sfile4
  File: sfile4 -> file.txt
  Size: 8         Blocks: 0          IO Block: 4096   symbolic link
Device: 259,2 Inode: 23364884    Links: 2
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/   fidir)   Gid: ( 1000/   fidir)
Access: 2024-03-23 20:53:24.412128593 +0700
Modify: 2024-03-23 20:38:10.584102797 +0700
```

```
Change: 2024-03-23 20:53:22.808128548 +0700
 Birth: 2024-03-23 20:38:10.584102797 +0700
```

The inode number is the same of `file2`. The following picture show the current scenario.

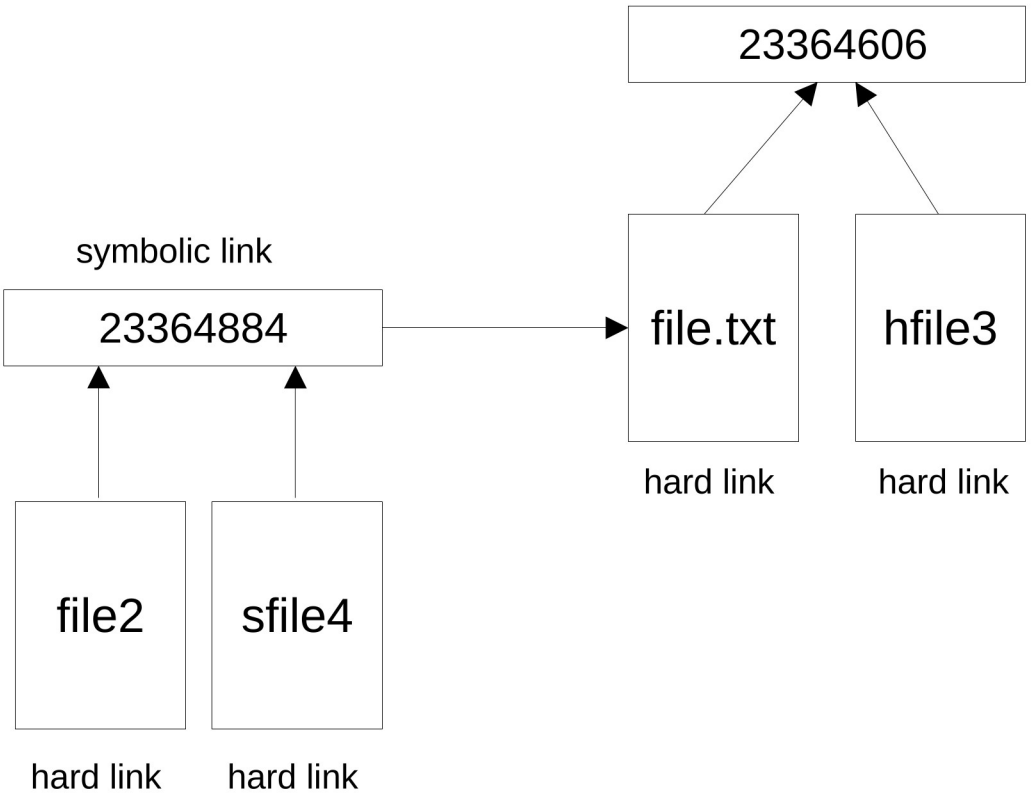

Fig 2. Relationship between symbolic and hard links

`sfile4` (like `file2`) is an hard link to a symbolic link, or better to an inode that is a symbolic link. Both the inodes 23364606 and 23364884 represent a file, but the second represents a file that is a pointer to another file. A symbolic link has no content, is only a pointer to the data of another file.

Let's view the content af all the files

```
[linux@fidir] cat *
hello
world
hello
world
hello
world
hello
world
```

We have four references to the same data with only two inodes.
The field `st_nlink` of the `stat` structure represent the number of the hard links of the file.

The field `st_mode` define the type and mode of the file.

On a Linux system, files can be of different types:

Regular file
: A file that contains data, it can be text or binary

Directory
: An access point in the file system containing other files and directories

Block file
: A file that provides buffered I/O access to devices with a fixed-size units of data

Character file
: A file that provides unbuffered I/O access to devices with variable-size units of data

FIFO
: Used for communication between processes, also called *named pipe*

Socket
: Used for network communication between processes and for non-network communication between processes on the same host (*interprocess communication*)

Symbolic link
: A file that points to another file

These types are specified in the `st_mode` field as listed in the following table.

`S_IFMT`	`0170000`	bit mask for the file type bit field
`S_IFBLK`	`0060000`	block device
`S_IFCHR`	`0020000`	character device
`S_IFDIR`	`0040000`	directory
`S_IFIFO`	`0010000`	FIFO
`S_IFLNK`	`0120000`	symbolic link
`S_IFREG`	`0100000`	regular file
`S_IFSOCK`	`0140000`	socket

Tab 4. Values of `st_mode`

The program `getinfo.c` retrieves information about the file supplied on the command line[22].

22 The code is taken from the `stat(2)` manual page

```c
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/sysmacros.h>
#include <time.h>

int main (int argc, char *argv[])
{
    struct stat sb;
    if (argc != 2) {
         fprintf(stderr, "Usage: %s <pathanme>\n", argv[0]);
         exit (EXIT_FAILURE);
    }
    if (lstat(argv[1], &sb) == -1) {
         perror("lstat");
         exit (EXIT_FAILURE);
    }
    printf("ID of device: [%x, %x]\n", major(sb.st_dev), \
         minor(sb.st_dev));
    printf("File type: ");
    switch(sb.st_mode & S_IFMT) {
         case S_IFBLK: printf ("block device \n"); break;
         case S_IFCHR: printf ("character device \n"); break;
         case S_IFDIR: printf ("directory \n"); break;
         case S_IFIFO: printf ("FIFO/pipe \n"); break;
         case S_IFLNK: printf ("symlink \n"); break;
         case S_IFREG: printf ("regular file \n"); break;
         case S_IFSOCK: printf ("socket \n"); break;
         default: printf("unknown? \n"); break;
    }
    printf ("I-node number: %ju\n", (uintmax_t) sb.st_ino);
    printf ("Mode: %jo (octal)\n", (uintmax_t) sb.st_mode);
    printf ("link count: %ju\n", (uintmax_t) sb.st_nlink);
    printf ("Ownership: UID=%ju GID=%ju\n", (uintmax_t) sb.st_uid,\
         (uintmax_t) sb.st_gid);
    printf ("I/O block size: i %jd bytes\n", (intmax_t) sb.st_blksize);
    printf ("File size: %jd bytes\n", (intmax_t) sb.st_blocks);
    printf ("Block allocated: %jd\n", (intmax_t) sb.st_blocks);
    printf ("Last staus change: %s", ctime(&sb.st_ctime));
    printf ("Last file access: %s", ctime(&sb.st_atime));
    printf ("Last file modifications: %s", ctime(&sb.st_mtime));
    exit (EXIT_SUCCESS);
}
```

---

getinfo.c Gets the information of the file

At line 9, the program declares a variable of type `struct stat` named `sb`, which will store the file information returned by `lstat`, and at line 10, it checks whether exactly one pathname argument was provided on the command line.
At lines 11–12, if the pathname is missing, we print a usage message to `stderr` and terminates using `exit(EXIT_FAILURE)`.
At line 14, the program calls `lstat`, passing the pathname provided by the user and the address of the `sb` structure, where the file metadata will be stored. Next, at lines 15–16, if `lstat` fails, we print the corresponding error message using `perror` and terminates with `EXIT_FAILURE`.
At line 18, the program prints the major and minor numbers of the device containing the file by using the functions `major` and `minor`[23] defined in `sys/sysmacros.h`.
At line 19, it starts to print the type of the file. The program performs a bitwise AND operation between `sb.st_mode` and the constant `S_IFMT`[24] at line 20, in order to extract the file type information from the `mode` field.
At lines 21–28, we use a `switch` statement to determine the file type and print the corresponding description, such as regular file, directory, symbolic link, socket, or device. Then, at line 30, we print the inode number stored in `sb.st_ino` after casting it to `uintmax_t`[25].
At line 31, the file mode in octal format is printed after casting `sb.st_mode` to `uintmax_t`.
At line 32, the number of hard links associated with the file is printed using the field `sb.st_nlink`.
After printing the user ID (UID) and group ID (GID) that own the file at line 33, the program prints the preferred I/O block size for the filesystem and at line 35 prints the size of the file.
At line 36, it prints the number of allocated blocks used by the file.
At lines 37–39, it calls `ctime`[26] to convert the timestamps stored in `sb.st_ctime`, `sb.st_atime`, and `sb.st_mtime` from `time_t` format into human-readable ASCII strings, and then prints them.

Let's execute the program passing the source file

```
[linux@fidir] ./getinfo getinfo.c
ID of device:          [103, 2]
File type:             regular file
I-node number:         23629952
Mode:                  100644 (octal)
link count:            1
Ownership:             UID=1000   GID=1000
I/O block size: i      4096 bytes
File size:             8 bytes
Block allocated:       8
Last staus change:     Sun Mar 24 22:06:09 2024
```

23 On Linux each device has a major and a minor number, the major number represents the device driver (IDE/SATA disk driver, floppy, serial port, etc..), the minor number identifies the device (many devices can share the same device driver)

24 `S_IFMT` is the bitmask for the file type, AND-ing with `sb.st_mode` return the file type. It is defined in `sys/stat.h`

25 `intmax_t` and `uintmax_t` are the signed and signed types to represent any value of any integer type supported by the implementation, see the `intmax(3type)` manual page. It is used for the inodes because this can have larger values. Theese types are defined in `stdint.h`

26 `ctime` takes a pointer as argument, is defined in `time.h`

```
Last file access:          Sun Mar 24 22:06:29 2024
Last file modifications:   Sun Mar 24 22:06:09 2024
```

Let's execute the program passing a device file

```
[linux@fidir] ./getInfo /dev/tty0
ID of device:              [0, 5]
File type:                 character device
I-node number:             13
Mode:                      20620 (octal)
link count:                1
Ownership:                 UID=0   GID=5
I/O block size: i          4096 bytes
File size:                 0 bytes
Block allocated:           0
Last staus change:         Wed Mar 20 17:57:16 2024
Last file access:          Wed Mar 20 17:57:16 2024
Last file modifications:   Wed Mar 20 17:57:16 2024
```

Permissions

On Linux system (and Unix systems) each file has a set of permissions that defines the access mode to the file.

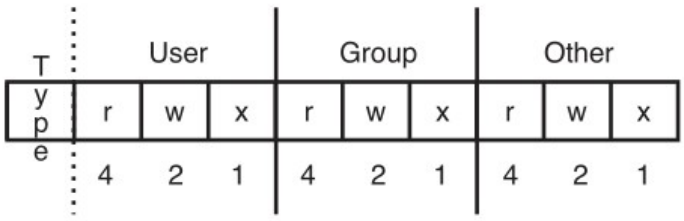

Fig 3. File permissions

Permission are divided in 3 categories: permissions of the Owner (user) of the file, permissions of the Group (of the owner and therefore of the file) and Other (other users). The first field defines the type of the file.

`-`	regular file
`l`	symbolic link
`b`	block device file
`c`	character device file
`d`	directory
`p`	FIFO (named pipe)

`s`	socket

Tab 5. File types

The second field represents the permissions: the first set (3 elements) defines the owner's permissions, the next the group's permissions, the last the others permissions. Each permission has a numeric and a string value.

`r`	read	4
`w`	write	2
`x`	execute	1

Tab 6. Types of permissions

Read permission means the user can read the contents of the file, write permission means the user can modify the file's contents, execute means the file can be executed. Concerning the directories: read means the user can view the directory's contents[27], write is the ability to add and delete files in the directory[28], execute means the user can traverse the directory (enter in the directory with the cd command)[29]. To add and delete files in a directory the user needs execute and write permissions on it.
The sum of these type specifies the permissions for the user/owner/other, so for example the owner can have a permission of 6 (6 = 4+2) that means read and write and no execute, the group can have a permission of `5` (5 = 4 + 1) that means read and execute but no write. All the possible combinations of 4,2 and 1 define a specific permission. The lowest permission is `000` that is defined by `---`. Where there is no permission, Linux use a − to represent it. The highest permission is `777`, where 7 is the sum of 4 + 2 + 1 which means read, write and execute.

The following table shows the possible combinations of values.

Total	**Sum**	**rwx**	**Permission**
`7`	4 + 2 + 1	`rwx`	read, write, execute
`6`	4 + 2	`rw-`	read and write
`5`	4 + 1	`r-x`	read and execute
`4`	4	`r--`	read only
`3`	2 + 1	`-wx`	write and execute
`2`	2	`-w-`	write only

27 To see the files attributes (as shown by `ls -l`) the execution permission is required for the directory

28 An user can delete a file in a directory if he has the write permission on that directory even if he is not the owner of that file

29 To execute `cd /var/log` the user must have the execution permission for all the directories of the path: `/`, `var` and `log`

1	1	--x	execute only
0	0	---	none

Tab 7. Combinations of the permissions values

The following table shows some examples.

777	user: read, write, execute; group: read, write, execute; other: read, write execute
664	user: red, write; group: read, write, other: read
755	user: read, write, execute; group: read, execute; oher: read, execute
5--	user: read, execute; group and others no permissions
-62	user: no permission; group: read and write; other: write

Tab 8. Example of permissions

To view permissions we can use `ls` with the option `-l`

```
[linux@fidir] ls -l
total 8
lrwxrwxrwx 2 fidir fidir  8 Mar 23 20:38 file2 -> file.txt
-rw-r--r-- 2      fidir fidir 12 Mar 23 21:09 file.txt
-rw-r--r-- 2      fidir fidir 12 Mar 23 21:09 hfile3
lrwxrwxrwx 2 fidir fidir  8 Mar 23 20:38 sfile4 -> file.txt
```

`file.txt` and `hfile3` are regular file with `644`, `file2` and `sfile4` are symbolic links with `777`.
In the previous example we executed our program passing the file `/dev/tty`, the output for the `st_mode` field was

Mode: 20620 (octal)

This means the file has the set of permission Owner: 6, Group: 2, Others. 0.

On Linux systems a regular file is represented in the first bit of the permissions with a `-`.
The permissions mask is a bitmask of 10 bit where the first bit represents the type of the file and the others 9 bits represent the three-bit fields of the owner, group and others[30]. The field `st_mode` of the structure `stat` is a bit mask containing the permissions and the type of the file. The file's type can be extracted by ANDing the value of the field with the constant `S_IFMT`. In the `getInfo.c` program we compared this values with the constants to print the type of file. To perform this operation are also available the following macros defined in `sys/stat.h`.

30 To change the permissions of a file see the manual page `chmod(1)`

Macro	Value
S_ISLNK(m)	(((m) & S_IFMT) == S_IFLNK)
S_ISREG(m)	(((m) & S_IFMT) == S_IFREG)
S_ISDIR(m)	(((m) & S_IFMT) == S_IFDIR)
S_ISCHR(m)	(((m) & S_IFMT) == S_IFCHR)
S_ISBLK(m)	(((m) & S_IFMT) == S_IFBLK)
S_ISFIFO(m)	(((m) & S_IFMT) == S_IFIFO)
S_ISSOCK(m)	(((m) & S_IFMT) == S_IFSOCK)

Tab 9. Macro to retrieve the type of file

The program `getfiletype.c` retrieves the type of the file supplied on the command line.

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
int main (int argc, char *argv[])
{
    struct stat sb;
    if (argc != 2) {
         printf("\nUsage: %s <file>", argv[0]);
         exit(EXIT_FAILURE);
    }
    if (lstat(argv[1], &sb) == -1) {
         perror("lstat");
         exit (EXIT_FAILURE);
    }
    int type = sb.st_mode & S_IFMT;
    if (S_ISREG(type))
         printf("\n%s: Regular file", argv[1]);
    else if(S_ISLNK(type))
         printf("\n%s: Symbolic link", argv[1]);
    else if(S_ISDIR(type))
         printf("\n%s: Directory", argv[1]);
    else if(S_ISCHR(type))
         printf("\n%s: Character device", argv[1]);
    else if(S_ISBLK(type))
         printf("\n%s: Block device", argv[1]);
    else if(S_ISFIFO(type))
         printf("\n%s: FIFO", argv[1]);
```

```
28    else if(S_ISSOCK(type))
29          printf("\n%s: Socket", argv[1]);
30    return 0;
31 }
```

---

getfiletype.c Gets the file type

At line 6, the program declares a variable of type `struct stat` named `sb`, which will store the file information returned by `lstat`.
At line 11, the program calls `lstat` to retrieve information about the file specified on the command line and stores the result in the sb structure.
At line 15, the program performs a bitwise AND operation between `sb.st_mode` and the constant `S_IFMT` to extract the file type information, and stores the result in the variable `type`.
At lines 16–29, the program checks the file type using macros such as `S_ISREG`, `S_ISLNK`, `S_ISDIR`, and others, then prints the corresponding file type description.

Let's execute the program.

```
[linux@fidir] ./getfiletype getfiletype.c
getfiletype.c: Regular file

[linux@fidir] ./getfiletype /home
/home: Directory

[linux@fidir] ./getfiletype /usr/bin/X
/usr/bin/X: Symbolic link

[linux@fidir] ./getfiletype /tmp/dbus-5HHov05X
/tmp/dbus-5HHov05X: Socket
```

By using the `stat` structure, we can develop programs for many different purposes. For example, we can write a program that searches for files with specific permissions, files with a given last modification date, or files whose size is greater or smaller than a specified threshold, and many other similar criteria.

Chapter II continues...

The competition

The field of Linux system programming is supported by several well-established and respected texts. While these works provide strong foundations, they often differ in scope, structure, and level of modernization. This book is designed to complement and extend these resources by addressing gaps in coverage, pedagogy, and practical application.

Advanced Programming in the UNIX Environment
This is one of the most influential and widely used texts in system programming. It provides a clear and rigorous introduction to UNIX system interfaces and remains a standard reference in both academic and professional contexts.

Strengths:

- Strong conceptual clarity and foundational coverage
- Well-structured explanations of core UNIX principles
- Widely trusted and historically significant

Limitations in the current context:

- Focuses on general UNIX concepts rather than Linux-specific behavior
- Limited coverage of modern Linux features and recent system interfaces
- Primarily conceptual, with less emphasis on runtime analysis and debugging workflows

Positioning of this book:
This book builds on the same foundational topics but focuses specifically on modern Linux systems, integrating newer features and emphasizing how programs behave in practice through tools such as tracing and debugging.

The Linux Programming Interface
A comprehensive and authoritative reference for Linux system programming, this book offers extensive coverage of system calls and library functions.

Strengths:

- Extremely detailed and comprehensive
- Linux-specific focus
- Excellent as a long-term reference

Limitations in the current context:

- Structured primarily as a reference manual rather than a guided learning path
- Less emphasis on building progressive understanding from fundamentals to advanced topics
- Limited integration of modern Linux developments such as newer asynchronous I/O models and evolving kernel features

Positioning of this book:
While maintaining a high level of technical depth, this book is designed as a structured learning journey, helping readers progressively develop mastery rather than navigate a reference. It also incorporates more recent Linux features and practices.

Other Resources (Online Documentation and Tutorials)
Developers often rely on:

- Linux manual pages
- online tutorials and blog posts
- fragmented educational resources

Strengths:

- Easily accessible and up-to-date
- Useful for quick reference and specific problems

Limitations:

- Lack of structure and continuity
- Inconsistent quality and depth
- Minimal guidance on how concepts connect or scale to real-world systems

Positioning of this book:
This book transforms these fragmented resources into a coherent and structured body of knowledge, enabling readers to move from isolated understanding to a complete system-level perspective.

Summary of Differentiation
This book differentiates itself by combining:

- **Modern Linux focus**
  Including features and practices not fully covered in existing classic texts
- **Structured progression**
  Moving from foundational concepts to advanced topics in a guided manner
- **Runtime-oriented approach**
  Emphasizing how programs behave using tools like *strace* and *gdb*
- **Bridging documentation and practice**
  Turning manual pages into practical learning tools through extensive examples

Contact Information

Full name: Marco Bruno
Address: Viale Giustiniano Imperatore 274, 00145, Rome, Italy
Email: mr.marco.bruno.2022@gmail.com
Tel: +855 963741727